Database Management Systems – Day 1

1. History of Databases

- Early computing used **File Management Systems (FMS)** in the 1960s–70s.
- Problems with FMS: redundancy, inconsistency, difficult access, poor security.
- **Database Management Systems (DBMS)** were created to solve these problems.
- Timeline Highlights:
    - 1960s: Hierarchical & Network databases (IBM IMS)
    - 1970s: Relational Database concept introduced by **E.F. Codd**
    - 1980s–1990s: Commercial RDBMS like Oracle, MySQL, SQL Server
    - 2000s+: Open-source DBMS & NoSQL systems

Key Point: Modern applications use **RDBMS** for structured data storage and management.

---

2. Data, Field, Record, and Database

- **Data:** Raw facts (`Alice`, `30`, `New York`)
- **Field:** Single data item (`Name`, `Age`)
- **Record:** Collection of fields (`Alice, 30, New York`)
- **Database:** Organized collection of related records, accessible & manageable

Example Table: Students

Name	Age	City
Alice	30	New York
Bob	25	Los Angeles

---

3. Limitations of File Management System (FMS)

- Redundant & inconsistent data
- Difficulty in accessing and updating data
- Poor security & no standard relationships
- Hard to maintain

Led to creation of **DBMS**.

---

4. What is a DBMS?

- A Database Management System (DBMS) is software that stores, manages, and retrieves data efficiently.
- Provides controlled access, security, and integrity.
- Examples: MySQL, Oracle, SQL Server, PostgreSQL.

Features of DBMS:

- Reduces redundancy
- Ensures data integrity
- Supports multiple users
- Provides backup and recovery
- Enforces security controls

---

5. Database vs DBMS

Feature	Database	DBMS
Definition	Collection of related data	Software to manage databases
Nature	Data	Software

Purpose	Store data	Manage, manipulate, and secure data
Example	Student Info, Bank Records	MySQL, Oracle

Memory Analogy: Database = memory/storage, DBMS = brain/software.

---

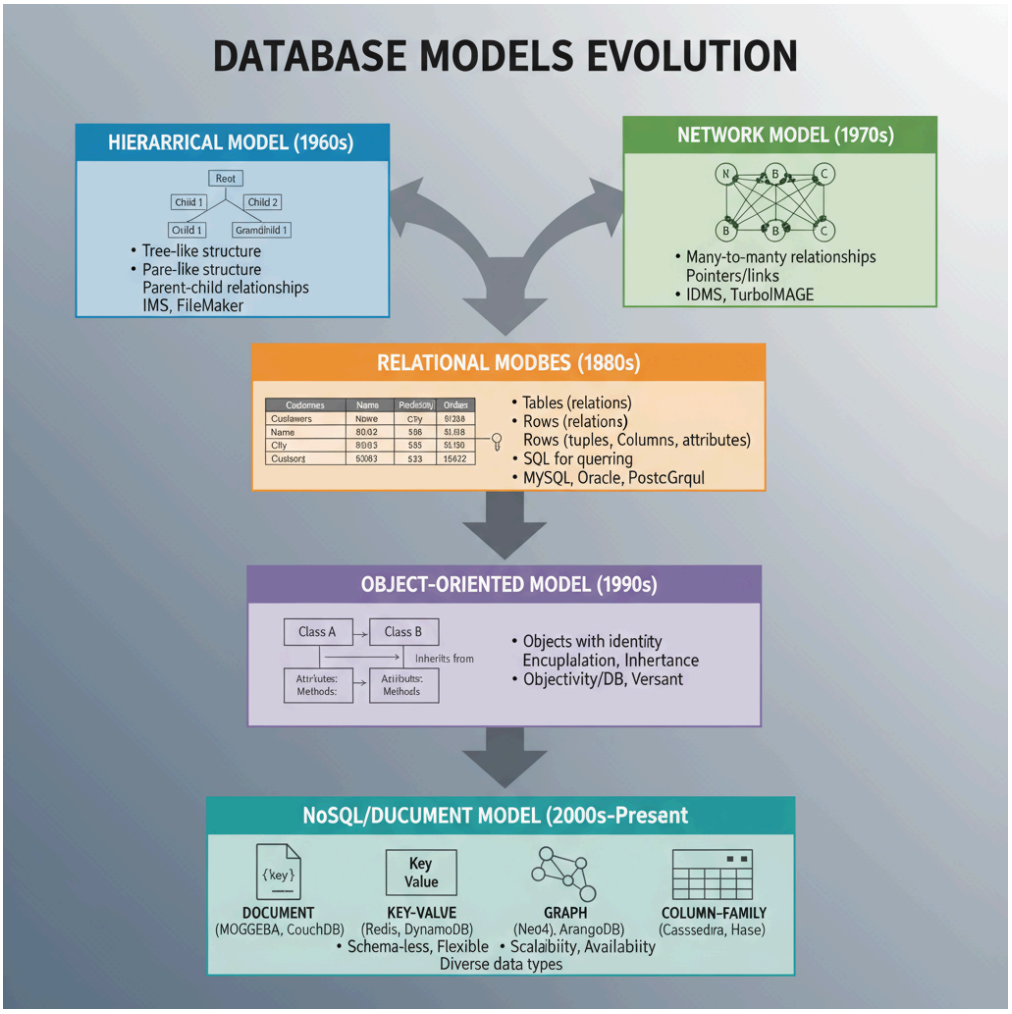
6. Database Models

Databases organize data in **different structures called models**.

Types of Database Models

Model	Description	Example
Hierarchical	Tree-like structure; parent-child relationships	IBM IMS
Network	Graph-like connections between records	IDMS
Relational (RDBMS)	Tables with rows & columns; uses keys	MySQL, Oracle
Object-Oriented	Data stored as objects with attributes & methods	PostgreSQL OO
NoSQL / Document	Stores unstructured/semi-structured data (JSON, key-value)	MongoDB, CouchDB

Most widely used today: Relational Model (RDBMS)



7.Components of a Database

Databases consist of several critical components that work together to **store**, **organize** and **retrieve data** effectively. Here is a detailed explanation of each component:

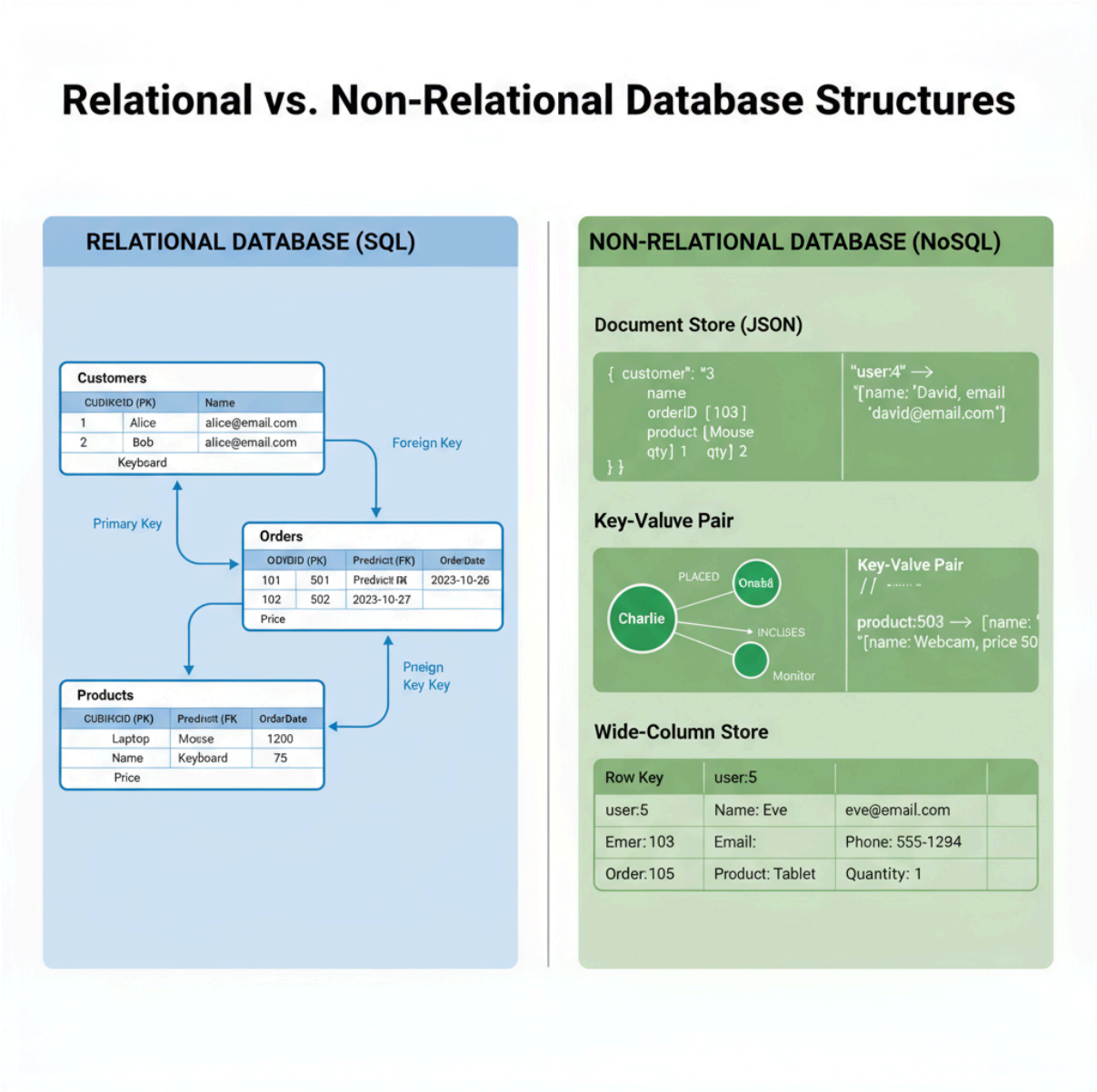
- **Data**: The actual information stored in the database, such as text, numbers, images, or files.
- **Schema**: The structural blueprint that defines how data is organized—tables, fields, data types, and relationships.

- **DBMS**: The software that manages database operations like storage, retrieval, and security (e.g., MySQL, Oracle).
- **Queries**: Instructions (usually SQL) used to retrieve or manipulate data within the database.
- **Users**: People or systems that interact with the database, each with specific roles and access permissions.

8.Types of Databases

Databases can be classified into two primary types Relational (SQL) and NoSQL Databases. NoSQL is then further divided into four types:

Document-oriented, Key-Value, Wide-Column, and Graph databases.



1. Relational Databases (RDBMS)

Relational databases organize data into tables made up of rows (records) and columns (fields). They use schemas (blueprints) to define how data is structured and how different tables relate to each other.

- Strict schema-based structure.
- Primary Keys (unique IDs) and Foreign Keys (relationships between tables).

- Strong ACID compliance (Atomicity, Consistency, Isolation, Durability).
- Ideal for structured data.

2. NoSQL Databases

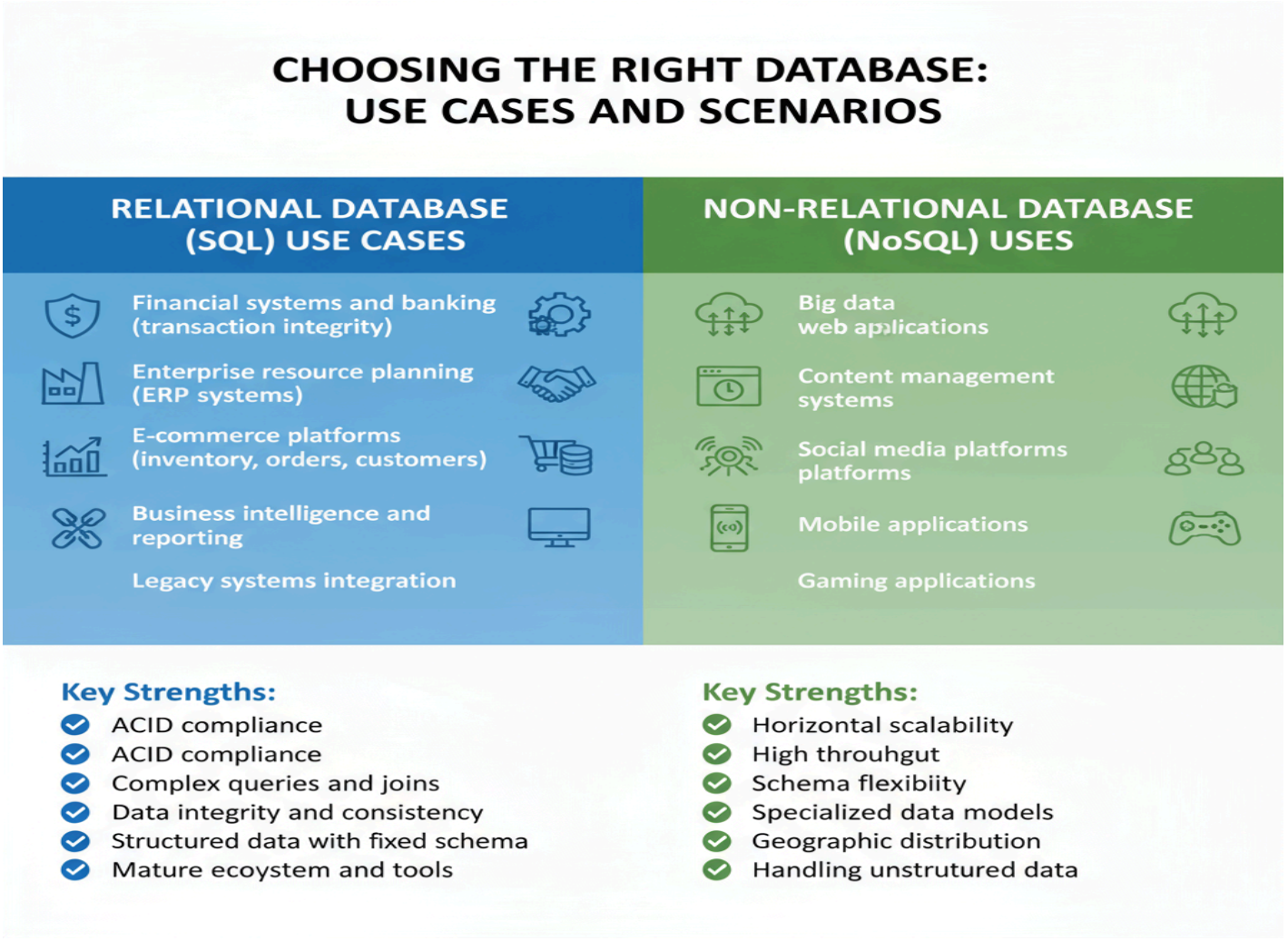
"NoSQL" stands for "Not Only SQL". These databases are designed to handle unstructured or semi-structured data, such as text, images, videos or sensor data. They don't rely on the traditional table format.

- Flexible data models (no fixed schema).
- Scales horizontally for high-volume data.
- Often optimized for specific use cases like graphs or time-series data.

9.ACID Properties

ACID stands for Atomicity, Consistency, Isolation, and Durability—four essential principles that ensure your database transactions are reliable, accurate, and secure.

- **Atomicity: Ensures transactions complete fully or not at all.**
- **Consistency: Ensures the database moves from one valid state to another.**
- **Isolation: Ensures that multiple transactions can happen at the same time without affecting each other.**
- **Durability: Saves changes permanently after completion.**



---

12. Database Design Process

1. Requirement Analysis – Identify what data is needed
2. Identify Entities – e.g., Student, Course
3. Identify Relationships – e.g., EnrolledIn

4. Draw E-R Diagram – Visualize entities & connections

An **Entity–Relationship (E-R) Diagram** is one of the most important tools in database design.
It provides a **visual blueprint** for how data will be stored, connected, and accessed.

Why E-R Diagrams are Important

- **Visual Representation:**
  Makes it easier to understand data flow and relationships between entities.

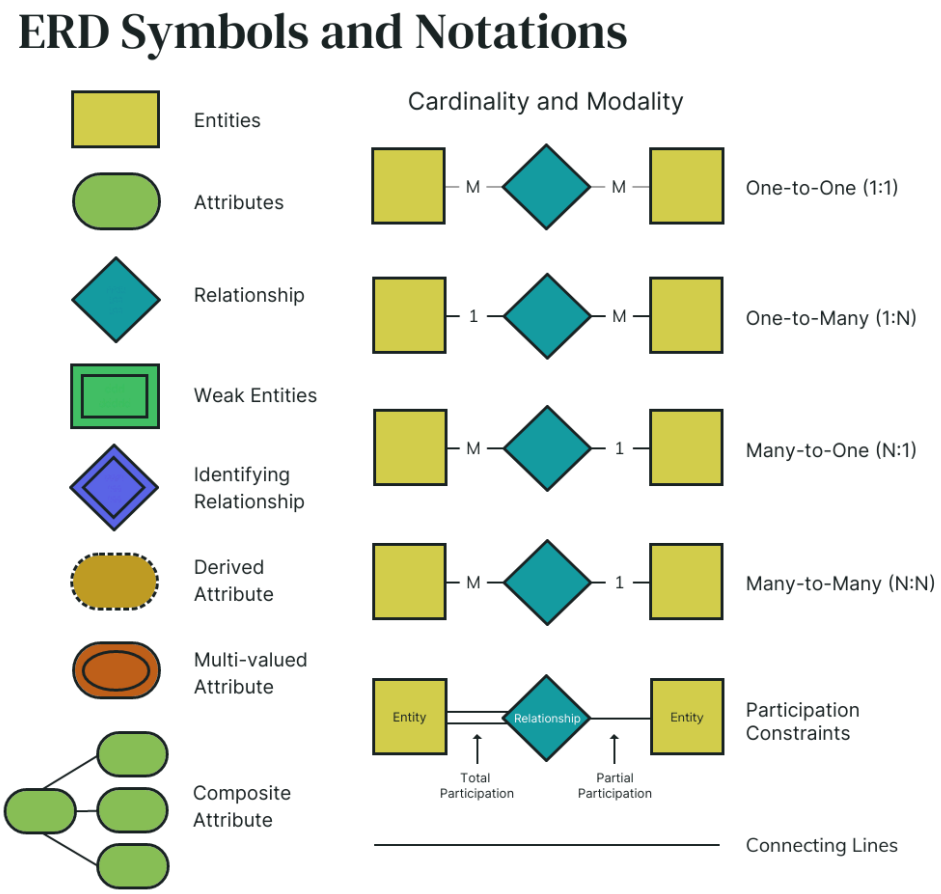
- **Clear Structure:**
  Defines how tables (entities) are connected using relationships and keys.

- **Helps in Communication:**
  Simplifies discussion between database designers, developers, and clients.

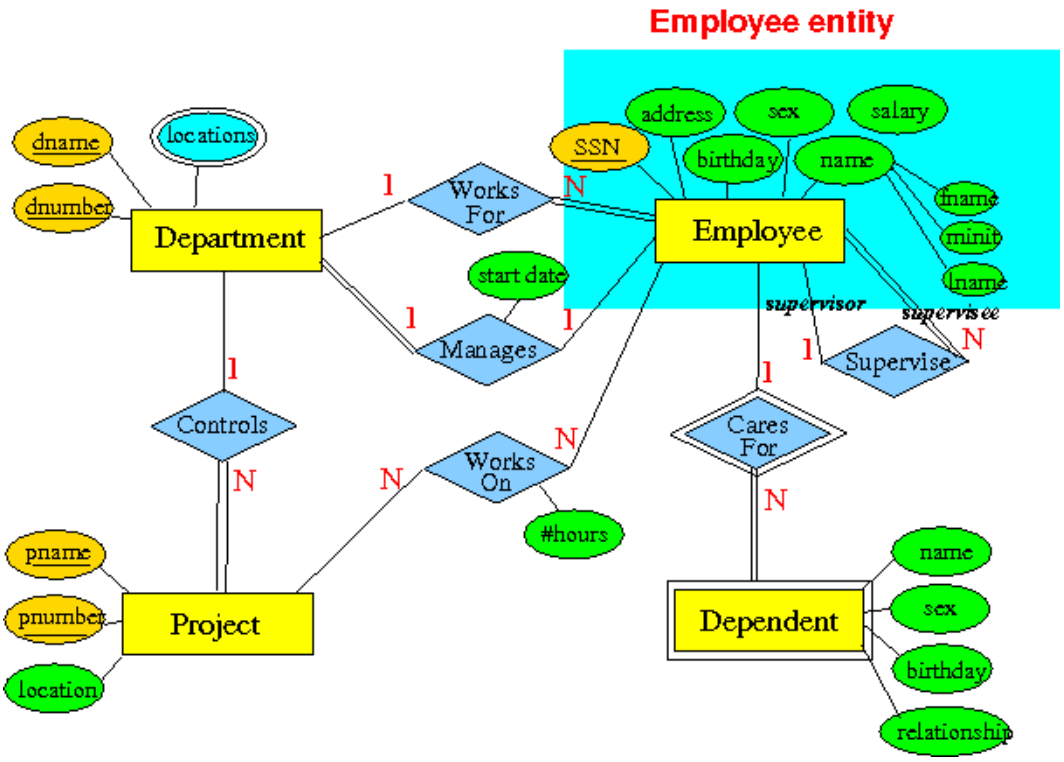
- **Prevents Redundancy:**
  Ensures that no data is duplicated across entities.

- **Foundation for Implementation:**
  Serves as the **first step before creating tables** in MySQL or any RDBMS.

- **Error Detection:**
  Helps identify missing entities or incorrect relationships early in design.



5. Convert E-R to Tables – Map entities and relationships



6. Normalization – Reduce redundancy & improve integrity
    - 1NF: Remove repeating groups
    - 2NF: Remove partial dependency
    - 3NF: Remove transitive dependency

Example: Student Courses Table (Before Normalization)

StudentID	Name	Courses
1	Alice	Math, Physics
2	Bob	Math, Chemistry

1NF (Remove repeating courses)

StudentID	Name	Course

1	Alice	Math
1	Alice	Physics
2	Bob	Math
2	Bob	Chemistry

2NF (Separate Student and Courses to remove partial dependency)
Student Table

StudentID	Name
1	Alice
2	Bob

Courses Table

StudentID	Course
1	Math
1	Physics
2	Math
2	Chemistry

3NF (Remove transitive dependency if any, e.g., city info in separate table)

StudentID	Name
1	Alice
2	Bob

CourseID	CourseName

101	Math
102	Physics
103	Chemistry

StudentID	CourseID
1	101
1	102
2	101
2	103

---

13. Evaluation / Importance of Databases

- **Data Integrity:** Accurate & consistent data
- **Reduced Redundancy:** Avoid duplicate data
- **Efficient Access:** Quick queries & reports
- **Multi-user Support:** Concurrent access without conflicts
- **Security & Backup:** Access control & recovery
- **Scalability:** Handle growing data and application needs

Summary: Databases are the backbone of all modern applications, from websites to banking systems.